Modul Lab 05 - Docker Core lokal dengan Windows PowerShell

COMP6991031 - Cloud Services | Topik RPS: Docker Core

Urutan ini memakai **PC Windows, Docker Desktop dengan Linux Engine, PowerShell, dan Chrome**. Semua command terminal berada di root repo **meet5CloudService**. Jalankan perintah satu per satu; baca hasilnya sebelum berpindah langkah. Tidak ada script pembungkus yang perlu dipasang atau dijalankan.

Kebijakan kelas: praktik merupakan latihan formatif, tanpa tugas, nilai, atau pengumpulan individu/per lab. Satu proyek besar tetap dikerjakan kelompok **3 orang**, dengan presentasi checkpoint minggu **7 (UTS)** dan hasil akhir minggu **14 (UAS)**. Catatan atau screenshot hasil sendiri boleh disimpan sebagai referensi proyek kelompok. Ketentuan resmi mengikuti RPS/LMS dan brief proyek kelompok.

Hasil belajar dan peta RPS

Subtopik Lab 05 pada RPS	Langkah pada modul	Hasil praktik
Menulis Dockerfile pertama untuk aplikasi Node.js/Python	2, memilih aplikasi Python yang disediakan	Dockerfile satu stage dan fungsi setiap instruksi
`docker build` dengan tag	3	Image `cloud-notes-api:single05`
Menjalankan custom image dengan environment variable	5-6	Container `cloudlab-core05`, APP_ENV, HTTP lokal
Multi-stage builder/runtime	4	Image `cloud-notes-api:lab05` dari dua stage
Push Docker Hub dan praktik `.dockerignore`	7 dan bagian akhir	Context terpilih serta tag/push ke namespace akun sendiri

Source app/main.py adalah API Cloud Notes **0.5**. API menyediakan `/health`, `/runtime`, GET/POST `/notes`, dan GET `/notes/{note_id}`. Latihan memilih jalur Python; Python dan dependency dipasang di image, sehingga tidak perlu memasang Python pada Windows untuk mengikuti langkah utama.

1. Siapkan PC dan buka satu repo lokal

1.1 Periksa Git dan Docker Desktop

Buka Docker Desktop dari Start dan tunggu Engine siap. Buka **PowerShell** pada folder pribadi yang dapat ditulis. Jalankan:

```
git --version
docker version
docker info --format '{{.OSType}}'
```

Fungsi: memastikan alat tersedia sebelum build. **Cara kerja:** Git menampilkan versinya; Docker CLI menghubungi daemon Docker Desktop. **Hasil yang dibaca:** `docker version` mempunyai bagian **Client dan Server**, dan command terakhir menghasilkan **linux**. Client tanpa Server berarti daemon belum siap. Engine atau instalasi yang dibatasi PC kampus ditangani bersama pengelola PC.

1.2 Clone dan masuk root repo

Untuk clone pertama, dari folder belajar:

```
git clone https://github.com/SeedFlora/meet5CloudService.git
Set-Location .\meet5CloudService
Get-Location
Get-ChildItem Dockerfile.single, Dockerfile, requirements.txt, app
```

Fungsi: menyiapkan source lokal. **Cara kerja:** `git clone` membuat folder repo; `Set-Location` memindahkan PowerShell ke folder itu. **Hasil:** lokasi berakhir pada `meet5CloudService` dan empat item ditemukan. Itulah **root repo** untuk seluruh command berikut. Tidak perlu `git init`.

Clone hanya dijalankan sekali. Bila folder repo sudah tersedia, buka folder tersebut di Explorer/VS Code lalu buka PowerShell di sana. Hindari membuat clone di dalam clone lain. Clone/pull base image dan instalasi dependency pada build pertama memerlukan internet. Selesaikan unduhan/build awal sebelum jam sesi gabungan 100 menit.

```
meet5CloudService/                <- PowerShell tetap di folder ini
  Dockerfile.single              <- latihan Dockerfile pertama
  Dockerfile                     <- versi builder + runtime
  .dockerignore
  requirements.txt
  app/
    __init__.py
    main.py
```

2. Tulis Dockerfile pertama

Buka file Dockerfile.single di editor. File sudah disediakan; untuk latihan menulis, isi/cocokkan seluruh baris berikut lalu simpan. **Blok ini adalah isi file Dockerfile, bukan command PowerShell.**

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
ENV APP_ENV=classroom
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Baris	Fungsi dan proses
`FROM python:3.12-slim`	Memilih base image yang sudah mempunyai Python. Tag menunjukkan keluarga versi/varian image.
`WORKDIR /app`	Menentukan direktori kerja untuk instruksi berikut dan proses aplikasi.
`COPY requirements.txt .`	Menyalin daftar dependency dari build context ke `/app`.
`RUN pip install ...`	Memasang paket saat **build** dan menghasilkan layer image. Tidak menjalankan API.
`COPY app ./app`	Menyalin source lokal menjadi `/app/app` dalam image.
`ENV APP_ENV=classroom`	Memberi nilai environment default pada image. Nilai dapat diganti saat `docker run -e`.
`EXPOSE 8000`	Mendokumentasikan port aplikasi. Port host tetap harus dipublish dengan `-p`.
`CMD [...]`	Perintah default ketika container **dijalankan**. Uvicorn mengimpor `app.main:app`.

COPY versus ADD: praktik ini memakai `COPY` karena inputnya berkas lokal. `ADD` mempunyai kemampuan tambahan, seperti ekstraksi arsip lokal dan sumber URL tertentu; kemampuan tambahan itu tidak diperlukan untuk menyalin `requirements.txt` atau `app/`. Pilih instruksi berdasarkan kebutuhan, bukan menganggap keduanya selalu sama.

RUN, CMD, dan ENTRYPOINT: `RUN` bekerja saat build. `CMD` memberi perintah/argumen default saat run; command lain yang ditulis setelah nama image dapat menggantikannya. `ENTRYPOINT` dapat menetapkan executable utama, dengan `CMD` sebagai argumen default. Dockerfile latihan ini memakai `CMD`, sehingga tidak perlu menambahkan ENTRYPOINT lain.

Checkpoint 1: Anda dapat menunjuk instruksi yang memasang dependency dan instruksi yang menjalankan server. `0.0.0.0` membuat server mendengarkan pada interface container; browser Windows akan memakai port host yang dipublish pada langkah 5.

3. Build image satu stage dan pahami tag

Tetap dari root repo:

```
docker build -f Dockerfile.single -t cloud-notes-api:single05 .
docker image ls cloud-notes-api
```

Fungsi: membuat image pertama dari Dockerfile yang baru dibaca. **Cara kerja:** `-f` memilih file Dockerfile; `-t` memberi nama/tag; titik terakhir berarti **build context root repo saat ini**. Dari context itulah `COPY requirements.txt` dan `COPY app` mengambil source. Docker mengunduh base/dependency ketika belum tersedia.

Hasil yang dibaca: proses build selesai tanpa error dan daftar image memuat repository `cloud-notes-api`, tag **single05**, IMAGE ID, serta SIZE. Baris `CACHED` pada build ulang berarti layer yang sesuai dipakai kembali. Image sudah tersedia, tetapi belum ada server HTTP hanya karena build selesai.

Tag adalah label image, bukan bukti bahwa image telah diunggah ke Docker Hub. `single05` dipakai untuk Dockerfile satu stage; `lab05` akan dipakai untuk versi multi-stage. Keduanya tetap berada di image store lokal sampai Anda melakukan push pada bagian akhir.

4. Build versi multi-stage: builder dan runtime

Buka Dockerfile pada root repo dan cocokkan dengan isi berikut. Blok ini juga **isi file**, bukan command terminal:

```
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

FROM python:3.12-slim AS runtime
ENV PATH="/opt/venv/bin:$PATH" \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
RUN useradd --create-home --uid 10001 appuser
WORKDIR /app
COPY --from=builder /opt/venv /opt/venv
COPY app ./app
USER appuser
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Fungsi: memisahkan tempat pemasangan dependency dan isi image yang dijalankan. **Cara kerja:**

1. Stage `builder` membuat virtual environment `/opt/venv` dan memasang dependency di sana.
2. Stage `runtime` mulai dari base image baru. `COPY --from=builder` mengambil hasil yang dibutuhkan dari stage pertama.
3. `PATH` memilih executable dalam venv; dua ENV Python mengatur cache bytecode dan output log.
4. Source `app/` ditambahkan ke runtime; `USER appuser` menjalankan server sebagai UID **10001**.
5. Stage terakhir adalah hasil default `docker build`. Stage builder tidak otomatis menjadi container layanan kedua.

Lab 05 · baca Dockerfile sebelum build

```
$ docker version --format '{{.Server.Version}}'
29.6.2

$ Select-String -Path Dockerfile -Pattern '^(FROM|RUN|COPY|USER|EXPOSE|CMD)'
Dockerfile:1:FROM python:3.12-slim AS builder
Dockerfile:3:COPY requirements.txt .
Dockerfile:4:RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt
Dockerfile:6:FROM python:3.12-slim AS runtime
Dockerfile:10:RUN useradd --create-home --uid 10001 appuser
Dockerfile:12:COPY --from=builder /opt/venv /opt/venv
Dockerfile:13:COPY app ./app
Dockerfile:14:USER appuser
Dockerfile:15:EXPOSE 8000
Dockerfile:16:CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Output aktual: FROM membentuk tahap image; RUN saat build; USER/CMD dipakai saat container dimulai.

Langkah: baca Dockerfile dan temukan dua `FROM`, `COPY --from`, `USER`, dan `CMD`. *Fungsi:* mengenali batas builder/runtime. *Proses:* hasil dependency dari builder disalin ke runtime. *Baca hasil:* dua stage, appuser, dan port container 8000. **Gambar historis** dari source Dockerfile ini; nomor versi Docker yang tercetak adalah contoh mesin penguji, bukan syarat versi yang harus sama di PC Anda.

Build versi runtime dan bandingkan image:

```
docker build -t cloud-notes-api:lab05 .
docker image ls cloud-notes-api
docker image inspect cloud-notes-api:single05 --format '{{.Size}}'
docker image inspect cloud-notes-api:lab05 --format '{{.Size}}'
```

Hasil yang dibaca: tag **single05** dan **lab05** tersedia. Dua command inspect menampilkan ukuran dalam byte. Catat angka dari PC sendiri; jangan menyalin ukuran screenshot.

Kunci perbandingan: multi-stage dapat mengurangi isi final ketika tool build/cache/source perantara tidak dibawa ke runtime. Pada contoh kecil ini kedua varian memakai keluarga base Python yang sama dan tidak memasang compiler besar; ukuran final **tidak dijamin lebih kecil** daripada satu stage. Pemisahan dependency/runtime dan user non-root tetap bisa dijelaskan dari Dockerfile. Bedakan ukuran image yang ditampilkan Docker dengan layer bersama yang tersimpan di disk.

Checkpoint 2: image `cloud-notes-api:lab05` berasal dari stage runtime. Source aplikasinya sama dengan versi satu stage; versi API tetap **0.5**, bukan versi library FastAPI atau versi image Python.

5. Jalankan image sendiri dengan environment dan port

Periksa nama container serta port yang sedang dipakai:

```
docker ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Ports}}'
```

Untuk langkah baru berikut, nama **cloudlab-core05** dan host port **18005** harus tersedia. Bila daftar menunjukkan nama/port itu sudah dipakai, lihat troubleshooting terlebih dahulu. Tidak ada command yang otomatis menghentikan atau menghapus container lain.

Jalankan image multi-stage yang baru dibuild:

```
docker run --name cloudlab-core05 -d -p 127.0.0.1:18005:8000 -e APP_ENV=classroom cloud-notes-api:lab05
docker ps --filter name=cloudlab-core05
docker logs --tail 15 cloudlab-core05
docker exec cloudlab-core05 id
docker image inspect cloud-notes-api:lab05 --format 'user={{.Config.User}}'
```

Bagian command	Fungsi dan hasil
`--name cloudlab-core05`	Nama container latihan yang dapat dicari saat membaca log/inspect.
`-d`	Menjalankan proses di background; terminal kembali menerima command.
`-p 127.0.0.1:18005:8000`	Port host **18005** diteruskan ke port **8000** dalam container, pada loopback PC.
`-e APP_ENV=classroom`	Memasok environment saat run. Nilai ini dibaca endpoint `/runtime`.
`docker logs --tail 15`	Membaca log Uvicorn. Tunggu server siap sebelum mengirim request.
`docker exec ... id`	Memeriksa user proses di container; hasil target UID 10001/appuser.

Image tidak berubah hanya karena dibuat satu container. `APP_ENV` adalah contoh konfigurasi lingkungan, bukan password. Nama tag pada akhir `docker run` menentukan image yang dipakai; pastikan **lab05**, bukan image dari latihan lain.



Langkah praktik ini: run `cloudlab-core05`, lalu baca ps, id, dan image inspect. *Fungsi:* membuktikan image/runtime/port yang dipilih. *Proses:* Docker menjalankan stage runtime dengan `USER appuser`. *Baca hasil:* UID 10001, tag lab05, dan mapping port. **Gambar historis memakai nama `cloudlab-api` serta host 8000 -> container 8000**. Pada panduan baru hasil Anda harus memakai **cloudlab-core05, host 18005 -> container 8000**; ukuran dan ID image dapat berbeda.

Tunggu log `Application startup complete.` sebelum mengirim request HTTP. Baca kembali `docker logs --tail 15 cloudlab-core05` untuk melihat perkembangan startup. Sesudah Uvicorn siap, uji endpoint melalui PowerShell pada blok terpisah berikut:

```
curl.exe -i http://127.0.0.1:18005/health
curl.exe -i http://127.0.0.1:18005/runtime
```

Fungsi: membuktikan aplikasi HTTP menjawab, bukan hanya proses Docker hidup. **Cara kerja:** request masuk ke port host 18005 lalu diteruskan ke Uvicorn port 8000. **Hasil:** kedua request HTTP **200**. Health memuat `status=ok` dan `service=cloud-notes-api`. Runtime memuat `environment=classroom` dan `storage=memory`.

`curl.exe` menyebut executable curl Windows secara eksplisit. `-i` menampilkan header beserta body. Up/Running tidak menggantikan pemeriksaan HTTP; health API ini hanya memeriksa proses aplikasi, belum mempunyai dependency database.

6. Pakai Swagger untuk melihat aplikasi yang dibuild

Buka **http://127.0.0.1:18005/docs** di Chrome. Halaman dokumentasi harus bertuliskan **Cloud Notes 0.5**.

1. Buka GET `/health` -> **Try it out** -> **Execute**. Baca status 200 dan JSON health.
2. Buka GET `/runtime`, jalankan Execute. Baca `environment=classroom` dan `storage=memory`.
3. Buka POST `/notes`, pilih Try it out, isi JSON berikut, lalu Execute:

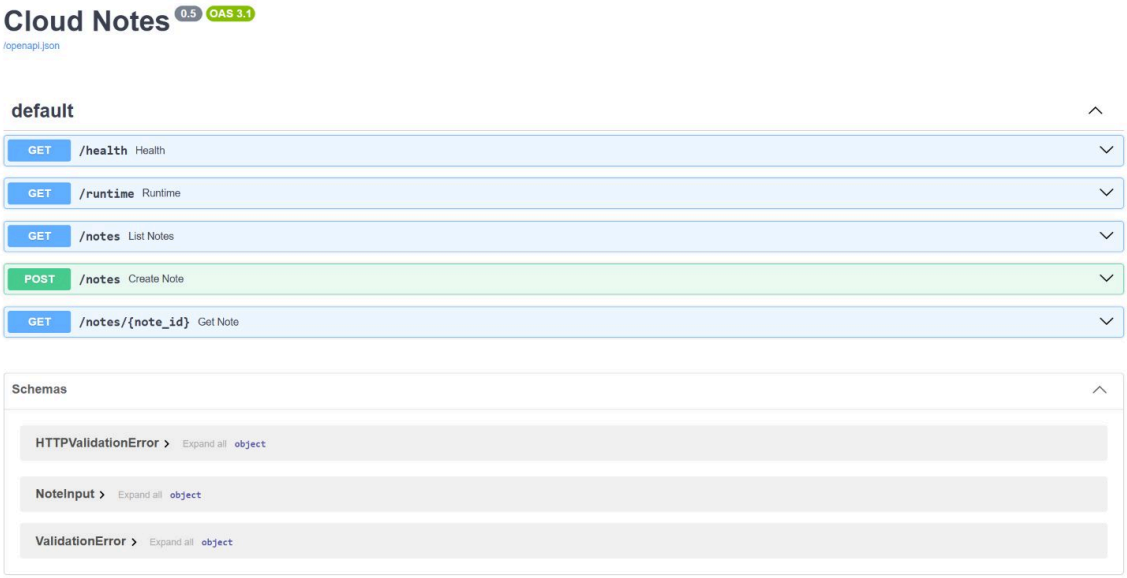
```
{"title":"Docker Core 05","content":"Image saya berjalan pada Docker Desktop lokal"}
```

4. Baca **201 Created**, judul, dan `id` yang diberikan API.

5. Buka GET `/notes` -> Execute. Catatan dengan ID/judul tersebut terlihat pada daftar.
6. Ulangi POST menggunakan JSON berikut untuk membaca validasi:

```
{"title":"   ","content":"uji input"}
```

Hasil: HTTP **422**, karena title hanya berisi spasi. Request invalid ditolak dan tidak menambah catatan. Gunakan Swagger untuk percobaan input salah ini; tidak ada blok penanganan error PowerShell yang harus diketik.



Langkah: buka **18005/docs** dan jalankan endpoint melalui Try it out/Execute. *Fungsi:* membaca kontrak dan mencoba UI aplikasi. *Proses:* FastAPI menghasilkan OpenAPI dari source API 0.5 yang sama. *Baca hasil:* versi 0.5 serta endpoint health/runtime/notes. **Screenshot UI aktual ini diambil dari aplikasi yang dijalankan native dengan Python pada port 18005; gambar ini tidak membuktikan Docker sedang berjalan.** Pada praktik Docker Desktop, periksa container cloudlab-core05 sendiri melalui ps, log startup, dan pemeriksaan user pada langkah 5. Python lokal tetap tidak diperlukan untuk mengikuti command Docker dalam modul.

GET daftar juga dapat dibaca dari PowerShell:

```
curl.exe -i http://127.0.0.1:18005/notes
```



Langkah: GET health/runtime, POST valid pada Swagger, lalu GET daftar di **18005**. *Fungsi:* menghubungkan environment dan respons aplikasi. *Proses:* satu proses API menyimpan catatan dalam dictionary memori. *Baca hasil:* storage memory, judul/ID pada POST dan GET. **Gambar historis menampilkan penyajian objek PowerShell dengan `value`/`Count`; body HTTP GET sebenarnya JSON array**. Command panduan baru menggunakan curl.exe agar body/header terlihat langsung.

Checkpoint 3: image sendiri berjalan sebagai container, environment terbaca, dan aplikasi merespons 200/201/422 sesuai request. Data API ini berada di memori proses. Ketika proses/container berhenti lalu dimulai lagi, catatan memori tidak menjadi bagian dari image dan dapat hilang. Persistensi database dipelajari pada Lab 06.

7. Periksa .dockerignore sebelum membagikan image

```
Get-Content .\.dockerignore
```

File latihan memuat:

```
.git
.venv
__pycache__
*.pyc
.env
.env.*
!*.example
README.md
```

Fungsi: mengecualikan file yang tidak perlu masuk build context. **Cara kerja:** Docker membaca pola `.dockerignore` saat menyiapkan context titik pada `docker build`. Pada Dockerfile ini `COPY` tetap memilih requirements dan folder app secara eksplisit. Pola `.env*` mencegah konfigurasi lokal/secret ikut context; pengecualian `!*.example` mengizinkan contoh konfigurasi yang memang aman.

Kunci: file yang diabaikan Git belum otomatis diabaikan Docker; keduanya membaca aturan berbeda. Jangan meletakkan password/token pada source `app/`, Dockerfile, atau nilai ENV di image. Dockerfile ini hanya memerlukan source aplikasi dan dependency; dokumen/screenshot latihan tidak perlu disalin ke image final. Mengubah `.dockerignore` tidak menghapus file host.

8. Tutup container latihan secara manual

Setelah selesai menggunakan **container yang dibuat pada langkah 5**, jalankan:

```
docker stop cloudlab-core05
docker rm cloudlab-core05
docker ps -a --filter name=cloudlab-core05
```

Fungsi: menghentikan proses dan melepas objek container latihan. **Cara kerja:** stop mengakhiri proses; rm menghapus container yang sudah berhenti. **Hasil:** nama cloudlab-core05 tidak lagi ada pada daftar; image single05/lab05 tetap tersedia. Tidak ada penghapusan semua container, image, atau volume PC. Command ini hanya dipakai untuk nama container yang Anda buat pada modul ini.

9. Docker Hub dari PC lokal - subtopik terakhir RPS

Bagian ini tetap termasuk materi Lab 05. Pada sesi gabungan **100 menit Docker Core + Compose**, lakukan bagian Hub **di luar alur utama 100 menit**, ketika akun dan internet siap. Tidak ada tugas atau nilai terpisah. Proses aplikasi tetap berjalan di PC lokal; Docker Hub dipakai sebagai registry penyimpan/distribusi image.

9.1 Siapkan namespace akun sendiri

Buka Docker Hub memakai **akun milik Anda**. Catat **username Docker Hub**, bukan email atau username GitHub. Pada akun/namespace yang Anda kuasai, buat repository bernama **cloud-notes-api** dan tentukan visibilitas sesuai arahan kelas. Jangan memakai namespace pengajar atau organisasi tanpa izin.

Sebelum menyalin command berikut, ganti setiap `namaakunanda` dengan username Docker Hub Anda yang sebenarnya. `namaakunanda` hanya penanda teks pada modul. Source aplikasi/konfigurasi contoh boleh dibagikan; periksa bahwa image tidak membawa data pribadi atau secret.

9.2 Login melalui Docker CLI lokal

```
docker login --username namaakunanda
```

Fungsi: memberi Docker CLI izin mengakses registry menggunakan akun Anda. **Cara kerja:** CLI meminta autentikasi secara interaktif. Ikuti permintaan Docker; gunakan personal access token pada prompt password bila pengaturan akun memerlukannya. Jangan menuliskan password/token pada command, screenshot, atau chat.

Hasil yang dibaca: pesan login berhasil untuk akun yang sesuai. Login tidak otomatis mengunggah image. Bila akun atau akses registry belum tersedia, pahami alurnya dan lakukan ketika akses siap; praktik build/run lokal tetap dapat dibaca tanpa akun Hub.

9.3 Beri tag yang mengandung namespace Docker Hub

```
docker tag cloud-notes-api:lab05 namaakunanda/cloud-notes-api:lab05
docker image ls namaakunanda/cloud-notes-api
```

Fungsi: memberi image lokal nama tujuan registry. **Cara kerja:** tag baru menunjuk image yang sama; tidak melakukan build ulang dan belum mengunggah layer. **Hasil:** repository `namaakunanda/cloud-notes-api`, tag `lab05`, dengan image ID yang cocok dengan image asal.

9.4 Push dan periksa hasilnya

```
docker push namaakunanda/cloud-notes-api:lab05
```

Fungsi: mengunggah layer/image ke repository milik Anda. **Cara kerja:** Docker memakai login lokal serta namespace/tag tujuan. Layer yang sudah ada dapat dipakai kembali oleh registry. **Hasil:** command selesai dan menampilkan digest hasil push. Buka repository **cloud-notes-api** pada akun Docker Hub sendiri, pilih tab Tags, dan pastikan **lab05** tercantum.

Kunci: nama image/tag lokal, nama tujuan namespace, dan digest mempunyai fungsi berbeda. Tag adalah nama yang dapat dipindahkan; digest mengidentifikasi isi image yang diterbitkan. Repo Git menyimpan source/riwayat, sedangkan registry menyimpan image. Keberhasilan push bukan bukti aplikasi aman atau mempunyai database persisten.

Tidak ada screenshot Docker Hub baru dari akun peserta pada materi ini. Bukti langkah Hub adalah output push/digest dan tag pada **akun Anda**, bila langkah tersebut benar-benar dikerjakan. Gambar GitHub pada materi lama tidak dipakai sebagai bukti Docker Hub.

10. Troubleshooting yang sesuai langkah lokal

Gejala	Yang diperiksa dan langkah berikutnya
Docker hanya menampilkan Client atau cannot connect	Buka Docker Desktop, tunggu Engine, ulangi `docker version`. Pengelola PC menangani instalasi/virtualisasi yang dibatasi.
Dockerfile/requirements/app tidak ditemukan	Periksa `Get-Location` dan `Get-ChildItem`. Terminal harus berada di root meet5CloudService, bukan folder app atau folder induknya.
Build gagal mengunduh base/paket	Periksa koneksi internet dan pesan pada langkah build yang gagal. Unduhan awal tidak termasuk percobaan offline.
Nama cloudlab-core05 sudah digunakan	Lihat `docker ps -a`; pastikan apakah itu container latihan Anda. Untuk menutup milik sendiri gunakan langkah 8, lalu ulangi run. Jangan menghapus nama container lain.
Port 18005 sudah digunakan	Lihat mapping pada `docker ps`. Koordinasikan proses yang menggunakan port itu; hentikan hanya milik sendiri yang sudah selesai.
curl belum mendapat respons sesudah run	Baca `docker logs --tail 15 cloudlab-core05`, tunggu Uvicorn siap, lalu ulangi health pada port 18005.
Browser membuka API yang berbeda	Cocokkan port 18005, nama cloudlab-core05, image lab05, dan label API 0.5 pada Swagger.
Ukuran multi-stage tidak lebih kecil	Baca batas contoh pada langkah 4. Bandingkan angka aktual dan isi stage; jangan mengarang persentase penghematan.
Docker Hub denied/unauthorized	Cocokkan akun login, username namespace, repository, dan izin push. Pastikan `namaakunanda` sudah diganti. Jangan mengganti dengan akun pengajar.

Kunci pemahaman untuk diskusi formatif

Pertanyaan	Kunci
Mengapa COPY requirements dilakukan sebelum COPY app?	Perubahan source app tidak selalu membatalkan cache pemasangan dependency bila daftar requirements tetap sama.
Mengapa RUN tidak menjalankan layanan saat kelas?	RUN dipakai ketika image dibuild; CMD menjalankan Uvicorn ketika container dibuat/dijalankan.
Apakah EXPOSE membuka port Windows?	Tidak. Mapping `-p 127.0.0.1:18005:8000` yang menghubungkan host dengan port container.
Apa yang dipindahkan dari builder ke runtime?	Venv `/opt/venv`; source app ditambahkan ke runtime. Runtime memilih appuser sebagai user proses.
Apa beda ENV pada Dockerfile dan -e saat run?	ENV memberi nilai default image; -e memasok/mengganti nilai untuk container tersebut.
Apakah tag berarti image sudah di registry?	Tidak. Tag lokal hanya label; `docker push namespace/image:tag` mengunggah ke registry setelah autentikasi/izin benar.
Mengapa .gitignore saja belum cukup?	Docker memakai .dockerignore untuk build context; Git memakai .gitignore untuk pelacakan source.

Tidak ada laporan atau pengumpulan baru. Gunakan hasil praktik untuk menjelaskan keputusan Dockerfile/image/container pada satu proyek kelompok.

Rujukan: Dockerfile, build image, multi-stage, build context dan .dockerignore, docker run, login, tag, push.